



8

14

1680 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Latches & Flip Flop

د. عماد الدين محمد



RB Informatics; 23/12/2024

الدارات المنطقية

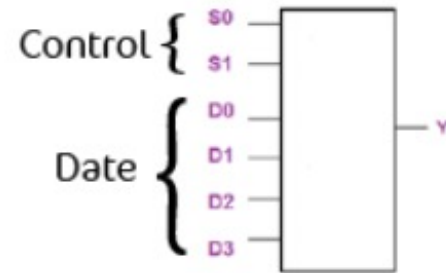
تحدثنا في المحاضرة السابقة عن الـ Encoder والـ Decoder، أما محتوى هذه المحاضرة فسيكون عن الـ Latches والـ Flip Flop وبعض الأفكار الأخرى..

Multiplexer

لنفرض لدينا تقاطع شوارع كما في الشكل المجاور، إن عدم وجود شرطي مرور ينظم العملية سيؤدي إلى أضرار وحوادث، فهو يسمح بمرور خط واحد فقط ويمنع مرور الباقي وهكذا حتى يسمح لجميع الخطوط بالمرور من الدوار دون حدوث أي مشاكل.

إن Multiplexer يلعب دور شرطي المرور فهو أيضا يقوم بتحديد أي دخل من المداخل الأربعة هو الذي سيخرج إلى المخرج الوحيد عن طريق S_1, S_2 (مفاتيح التحكم)، حيث تأتي المعلومات عبر المداخل الأربعة ويقوم الـ MUX بتحديد أي من هذه المداخل الأربعة سيظهر على الخرج (أي أنه في وقت واحد يمر لدخل واحد فقط للخروج).

$S_1 = 0, S_2 = 0 \rightarrow D_0$ هو الذي سيخرج
 $S_0 = 0, S_1 = 1 \rightarrow D_1$ هو الذي سيخرج
 $S_0 = 1, S_1 = 0 \rightarrow D_2$ هو الذي سيخرج
 $S_0 = 1, S_1 = 1 \rightarrow D_3$ هو الذي سيخرج



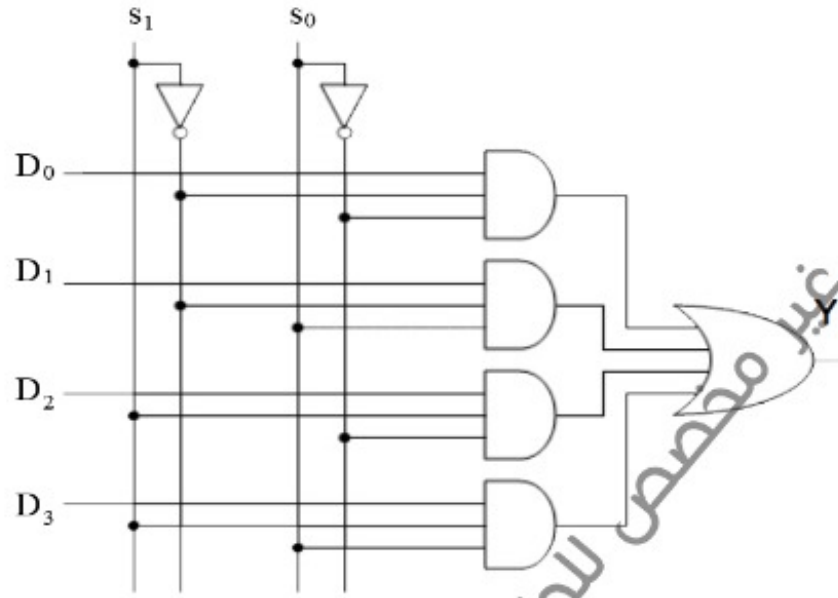
S1	S0	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

جدول الحقيقة لهذه الدارة:

$$Y = D_0 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_1 S_0 + D_2 S_1 \bar{S}_0 + D_3 S_1 S_0$$

تفسير العلاقة Y:

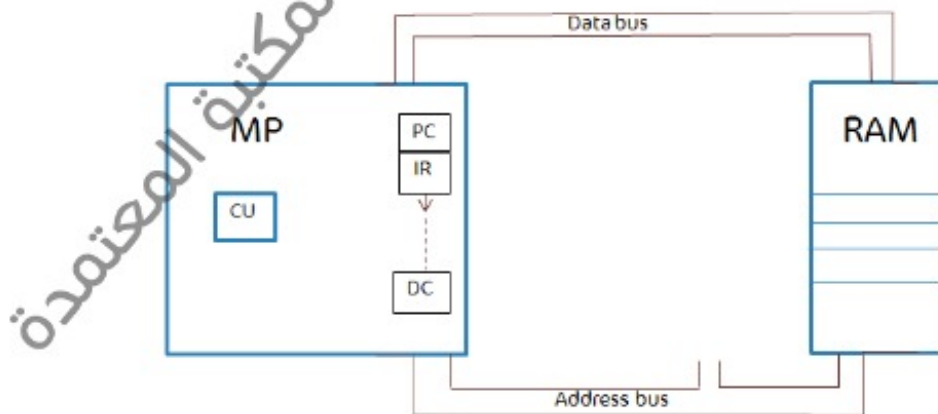
عندما $D_i = 0$ حيث $i = 0, 1, 2, 3$ فكل حد يحوي D_i سيحذف من علاقة Y.
 عندما $D_i = 1$ حيث $i = 0, 1, 2, 3$ فكل حد يحوي D_i لن يتأثر بشيء في علاقة Y.



الشكل السابق يوضع داخل صندوق الـ MUX

استخدام MUX

يستخدم MUX في جهاز الكمبيوتر حيث يحدد أي جهاز من الأجهزة الموصولة ستعمل ، فمثلاً إذا كان المطلوب هو إصدار أمر على الشاشة فالـ MUX سيفعل فقط مسار الشاشة وإذا كان الأمر استقبال معلومات من الكيبورد فإنه سيفعل فقط مسار الكيبورد وهكذا، أي أنه ينظم عملها عن طريق اختيار جهاز من بين عدة أجهزة موصولة بالكمبيوتر كي لا تتداخل المعطيات مع بعضها البعض تماماً كمثال شرطي المرور.



تصنف الدارات المنطقية إلى نوعين رئيسيين:

1. دارات منطقية توافقية Combinational Logic Circuits وفيها وجدنا أن خرج الدارة في أي لحظة زمنية يعتمد على الدخل المطبق في تلك اللحظة ولا يوجد Feedback .
 2. دارات منطقية تعاقبية Sequential Logic Circuits وفيها يعتمد خرج الدارة على الدخل المطبق وعلى الخرج السابق لهذه اللحظة.
- تعرفنا في المحاضرات السابقة على عدة دارات من النوع Combinational Logic Circuits وستتعرف الآن على دارات من النوع Sequential Logic Circuits.

Latches

وهي عبارة عن دارة تعاقبية Sequential Logic Circuit.

والتي تتميز بخاصية الذاكرة (تخزين مؤقت) (Temporary storage element) تعمل على تخزين 1bit فقط (bistable storage element) حيث نقصد بكلمة bistable (two stable states) أي حالتين فقط صفر أو واحد، وستتعرف على أول Latches:

SR Latches

تمثيل SR Latches باستخدام بوابات NOR:



عندما $Q = 0$ يكون $Q^- = 1$ ، والعكس بالعكس.

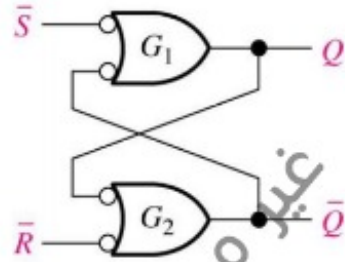
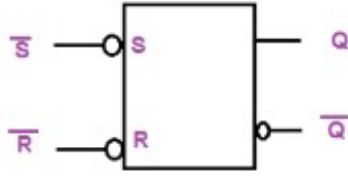
مجرد وجود سهم عائد من الخرج إلى الدخل أي يوجد Feedback فهي Sequential Circuit.



Good things take time..

ويمكن تمثيل الـ SR- Latches باستخدام بوابات أخرى:

بوابات OR مع inverted inputs:



S	R	Q^t	Q^{t+1}
0	0	0	0 (NC)
0	0	1	1 (NC)
0	1	0	0 (Reset)
0	1	1	0 (Reset)
1	0	0	1 (Set)
1	0	1	1 (Set)
1	1	0	X (Invalid)
1	1	1	X (Invalid)

عندما $R=1$ نكون بحالة Reset أي ان الخرج صفر مهما كان الدخل.

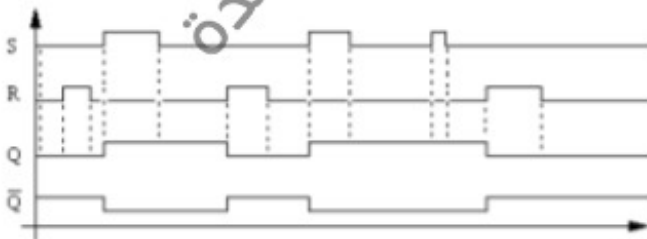
عندما $S=1$ نكون بحالة Set أي أن الخرج واحد مهما كان الدخل.

عندما $S=0$ $R=0$ نكون بحالة No Change أي أن الخرج لا يتغير ويبقى بحسب قيمة Q^t

عندما $S=1$ $R=1$ نكون بحالة عدم تعيين Invalid.

وعندما تكون الـ $S = 1$, $R = 1$ تظهر حالة عدم تعيين أي تكون $Q = \bar{Q}$ تسمى حالة عدم تعيين.

Timing Diagram:



$$S = 0, R = 0 \rightarrow Q = 0$$

لأن الحالة السابقة صفر

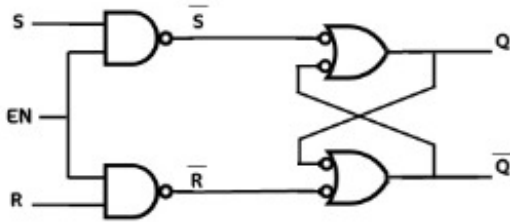
$$S = 0, R = 1 \rightarrow Q = 0$$

$$S = 1, R = 0 \rightarrow Q = 1$$

ملاحظة: يوجد IC جاهز بالسوق باسم quad SR- Latches.

لو أجرينا تغيير على تمثيل الـ SR- Latches بإضافة Enable فإننا نحصل على Latches أخرى تسمى: Gated SR Latch

Gated SR Latcher



نلاحظ عندما:

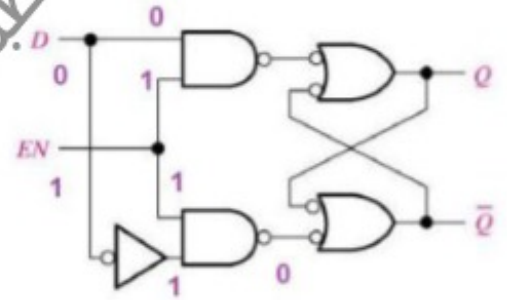
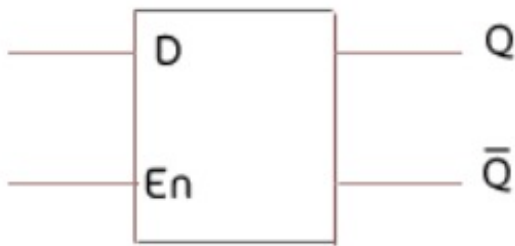
$En = 1 \rightarrow RS \text{ works}$

$En = 0 \rightarrow RS \text{ doesn't work}$

ولو أجرينا التغيير التالي فإننا نحصل على Latches أخرى تسمى

Gated D Latch

Gated D Latcher



نلاحظ أن D هي دائرة Transparent، أما إذا كان $Enable=0$ أي لا تعمل الدارة فسيكون الخرج هو نفسه Q^t مهما كانت الـ D، في حالة $Enable=1$ سيكون الخرج نفسه D.

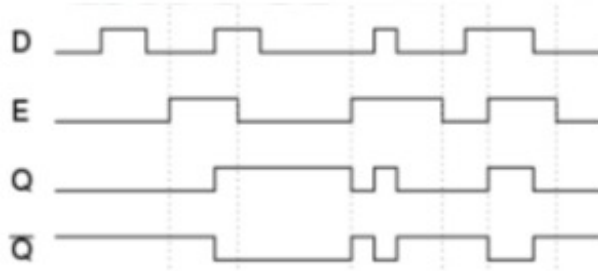
أي أن $D = Q^{t+1}$ كما في الجدول الجانبي:

D	Q^t	Q^{t+1}
0	0	0
1	0	1
0	1	0
1	1	1

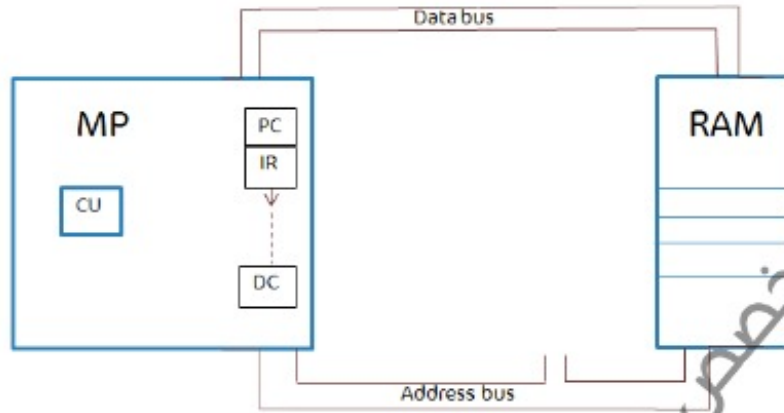
لذا نقول أن D هو Mirror (مرآة) لـ Q^{t+1} ، فما نطبقه على D نحصل عليه في Q^{t+1} .

ملاحظة: توجد جاهزة باسم 74ls75 quad D Latches.

Timing Diagram:



استخدام ال Latches



تستخدم للتخزين المؤقت عندما يكون data bus الموصول بين المعالج والرام مشغول أي يقوم بنقل المعلومات ويوجد معلومات أخرى تريد أن تمر من هذا الطريق فهذا تأتي أهمية latch حيث يقوم بتخزين المعلومات بشكل مؤقت ريثما ينتهي data bus من انشغاله بالمعلومات السابقة.

في الفقرة التالية سنبدأ بال Flip Flop، ولكن قبل البدء بالحديث عنها يجب أن نعرف الفرق الجوهرى بينها وبين ال Latches، وهو أن القلاب Flip Flop يكون متزامن (synchronous) ويستخدم clock للدقة، أما في ال Latches، فهذا نهتم بال Enable.

Flip Flop

بما أنه متزامن أي سنعتمد على ال clock، سنذكر بها:
ال clock هو عبارة عن train of pulse (قطار نبضات)

حيث أن في كل نبضة يوجد leading edge (جبهة صاعدة) و falling edge (جبهة هابطة).
يوجد FLIP FLOP يعمل عند الجبهة الصاعدة ومنها من يعمل عند الجبهة الهابطة.



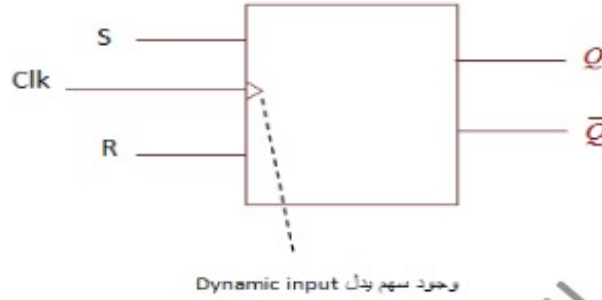
يعمل عند الجبهة
الهابطة



يعمل عند الجبهة
الصاعدة

لدينا عدة أنواع من FLIP FLOP وستتعرف على كل منها :

RS Flip-Flop



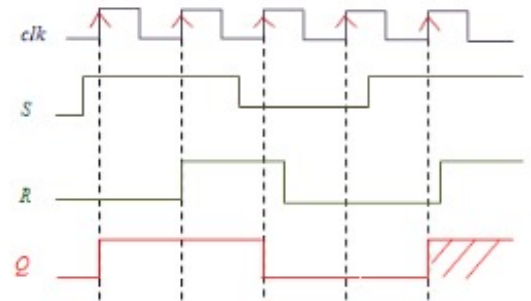
جدول الحقيقة لهذه الحالة:

Clock	S	R	Q^t	Q^{t+1}	
↑	0	0	0	0	No change (NC)
↑	0	0	1	1	
↑	0	1	0	0	Reset
↑	0	1	1	0	
↑	1	0	0	1	Set
↑	1	0	1	1	
↑	1	1	0	X	Invalid
↑	1	1	1	X	

اعتمدنا هنا على ال Leading edge الجبهة الصاعدة (أي الانتقال من ال 0 لل 1) وذلك بوضعنا سهم نحو الأعلى.

Timing Diagram:

نقوم بداية بتعيين نقاط التي ستحدث عندها التغيير (الموجة الصاعدة أو الهابطة) وسنشرح على الجبهة الصاعدة، ننزل خطوط من كل جبهة صاعدة من clk (أو جبهة هابطة إذا كان يعمل عند الجبهة الهابطة) حيث افترضنا أن القيمة الابتدائية لـ Q هي الصفر.



إن كان يعمل عند الجبهة الصاعدة:

عند إشارة الجبهة الصاعدة و عندما $R=1$ نكون بحالة Reset أي أن الخرج صفر مهما كان الدخل.
عند إشارة الجبهة الصاعدة و عندما $S=1$ نكون بحالة Set أي أن الخرج واحد مهما كان الدخل.

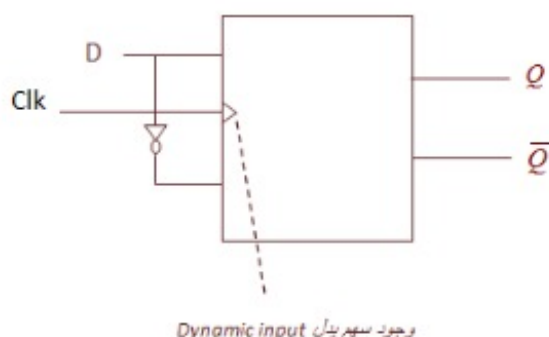
عند إشارة الجبهة الصاعدة و عندما $S=0, R=0$ نكون بحالة No Change أي أن الخرج لا يتغير ويبقى بحسب قيمة Q^t .
عند إشارة الجبهة الصاعدة و عندما $S=1, R=1$ نكون بحالة عدم تعيين Invalid.

ملاحظة: نفس الأمر بالنسبة للجبهة الهابطة إلا أنه في الهابطة نحدث التغير عند وصول الجبهة الهابطة.

D Flip-Flop

جدول الحقيقة لهذا النوع:

clk	D	Q^t	Q^{t+1}
↑	0	0	0
↑	0	1	0
↑	1	0	1
↑	1	1	1



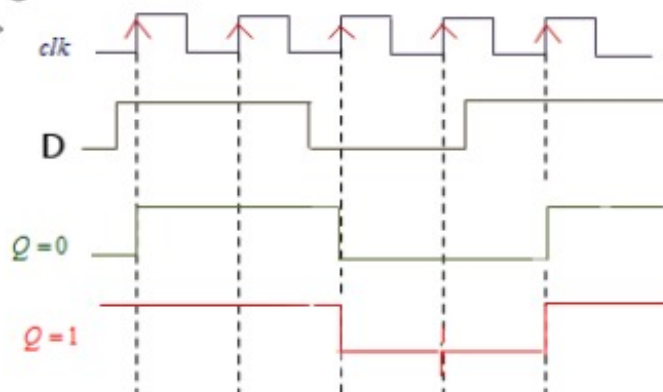
وجود سهم يدل Dynamic input

سنشرح في حال كان يعمل عند الجبهة الصاعدة:

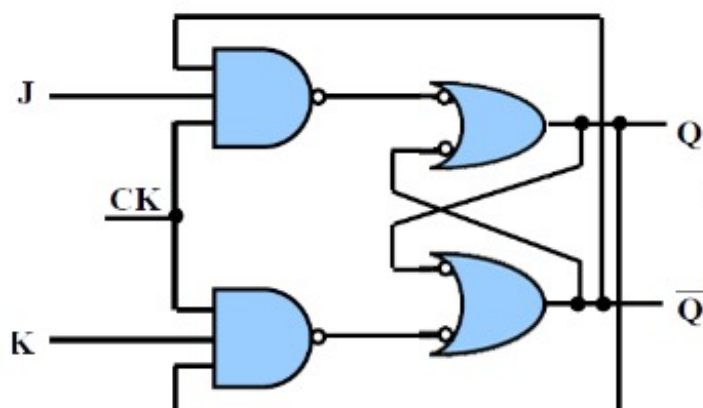
التغير سيحدث عند الجبهة الصاعدة فقط وسيكون سيكون الخرج نفسه D أي أن $D = Q^{t+1}$ كما هو موضح في جدول الحقيقة.

لذا نقول أن D هو Mirror (مرآة) Q^{t+1} فما نطبقه على D نحصل عليه في Q^{t+1} ولكن عند وصولنا للموجودة الصاعدة (أو الهابطة إن كان يعمل عند الهابطة).

Timing Diagram:



JK Flip-Flop



سمي بهذا الاسم نسبةً للعالم (Jack Kilby) مخترع IC. يعد هذا النوع من أهم أنواع القلايات كونه حل مشكلة عدم التعيين التي وجدناها في القلاب RS عندما (S=1, R=1).

J	K	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

No change (NC)

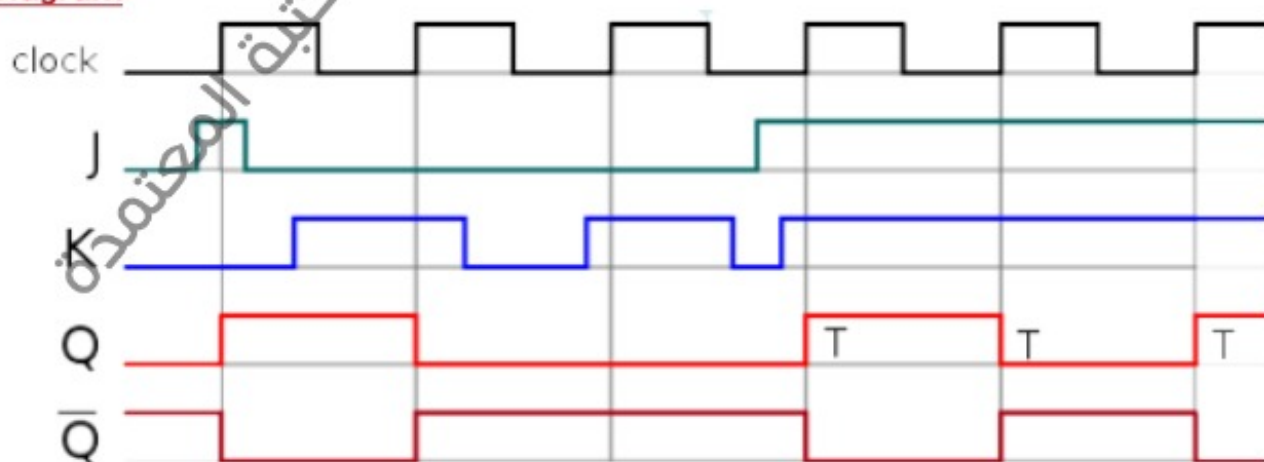
Reset

Set

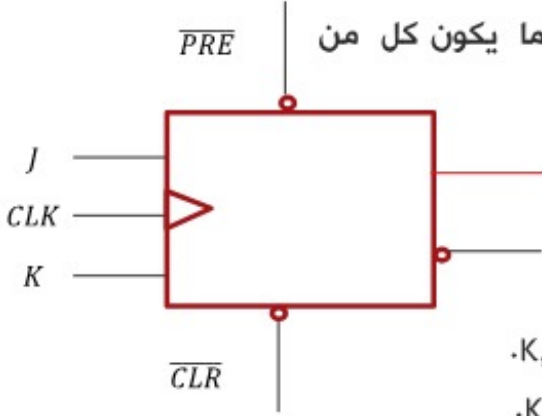
Toggle

نلاحظ أن هذا القلاب حل مشكلة عدم التعيين حيث عندما يكون (J=1, K=1) فإن Q^{t+1} يكون عكس Q^t .

Timing Diagram:



A synchronous Inputs ($\overline{Preset}$, $\overline{CLR}$)



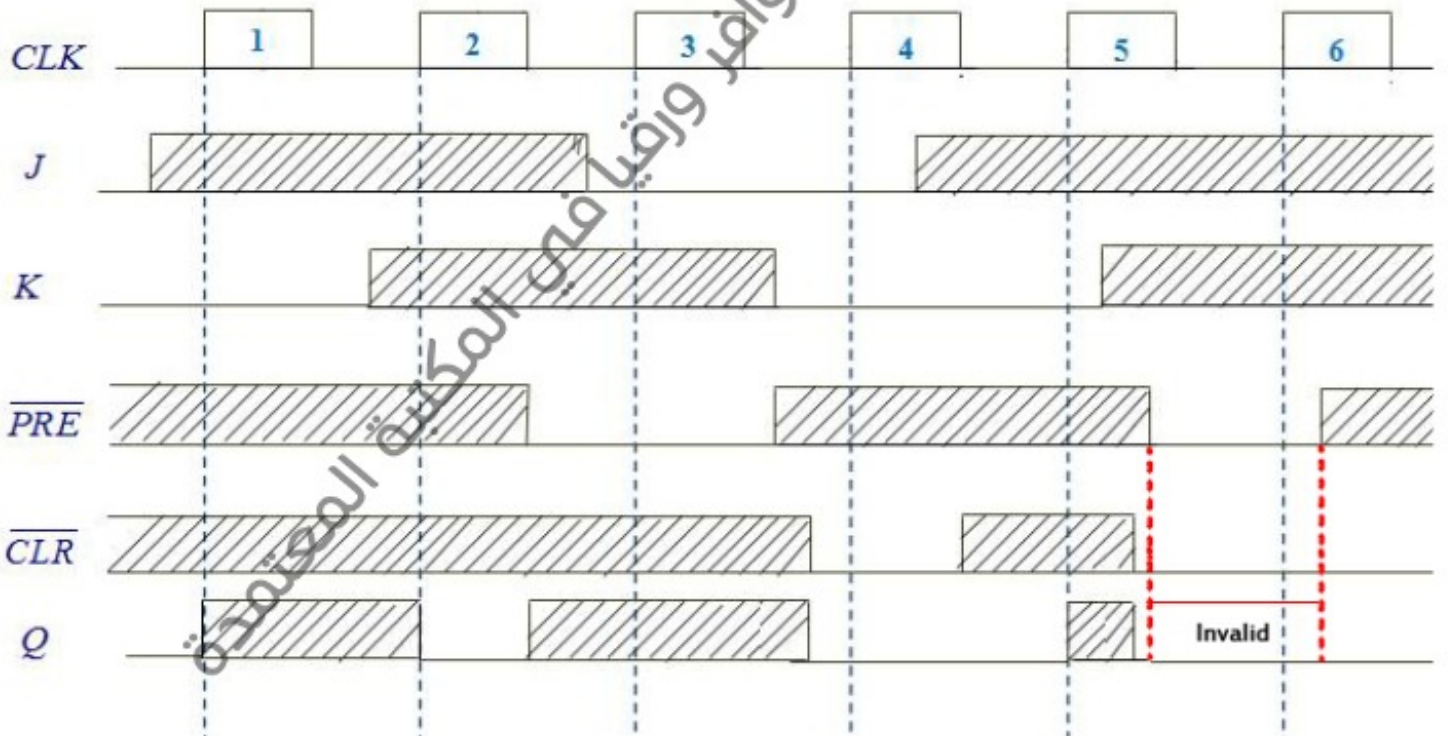
الغير متزامن أي أنه غير مرتبط بـ clock وهو يعمل عندما يكون عندما يكون كل من $\overline{CLR}$, $\overline{Preset}$ له القيمة low أي عند الصفر.

$\overline{PRE}_{active} = 0 \Rightarrow Q = 1$ بغض النظر عن قيمة $K, J, CLOCK$.

$\overline{CLR}_{active} = 0 \Rightarrow Q = 0$ بغض النظر عن قيمة $K, J, CLOCK$.

وعندما $\overline{CLR}, \overline{PRE} = 1$ (nonactive) فإنه يتحول إلى قلاب J, K والذي تعرفنا عليه حيث يكون الخرج بحسب J, K . ولا يمكن أن يكون كلاهما فعال $\overline{CLR}, \overline{PRE} = 0$ (active).

Timing Diagram:



لاحظ منطقة invalid حيث $\overline{CLR}, \overline{PRE} = 0$ (active).

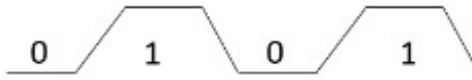
FF _ Characteristics

تذكرة:

إن شكل الموجة المثالية Ideal للنبضات موضح جانباً:



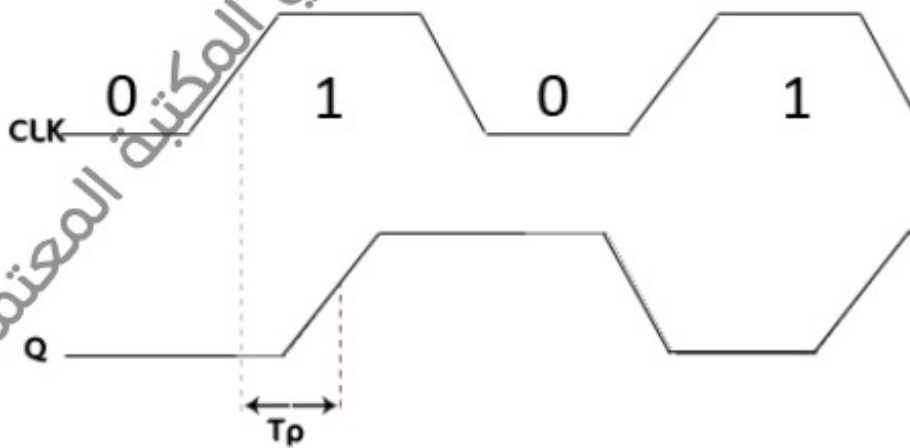
ولكن في الواقع لا يمكن أن تنتقل النبضة من 0 إلى 1 أو العكس بشكل آني أما الشكل الحقيقي Real للدارة فيكون كما في الشكل المجاور:



Real pulse

1. propagation delay time:

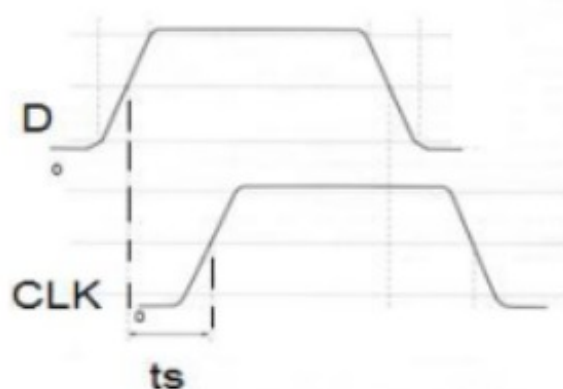
وهو الفارق الزمني الذي يتأخر فيه الحصول على الخرج حيث أن النبضات في الواقع هي ليست مثالية لذا فإنه يوجد فارق زمني بين تطبيقنا للصفر أو الواحد و الحصول على الخرج المناسب ويرمز لهذا الفاصل الزمني بـ T_p .



لذا نعرفه بأنه هو الزمن اللازم لتغيير قيمة الخرج Q بعد وصول النبضة.

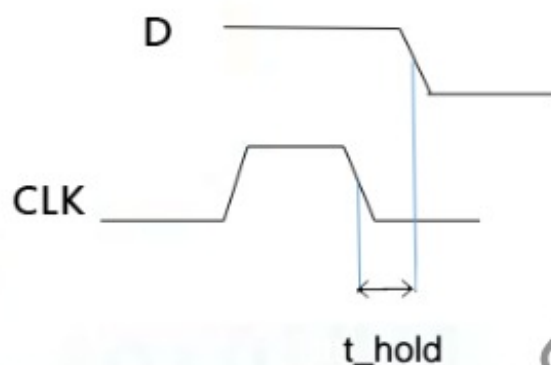
ملاحظة: عندما يكون T_p كبير يتأثر أداء الدارة.

2. Set-Up time:

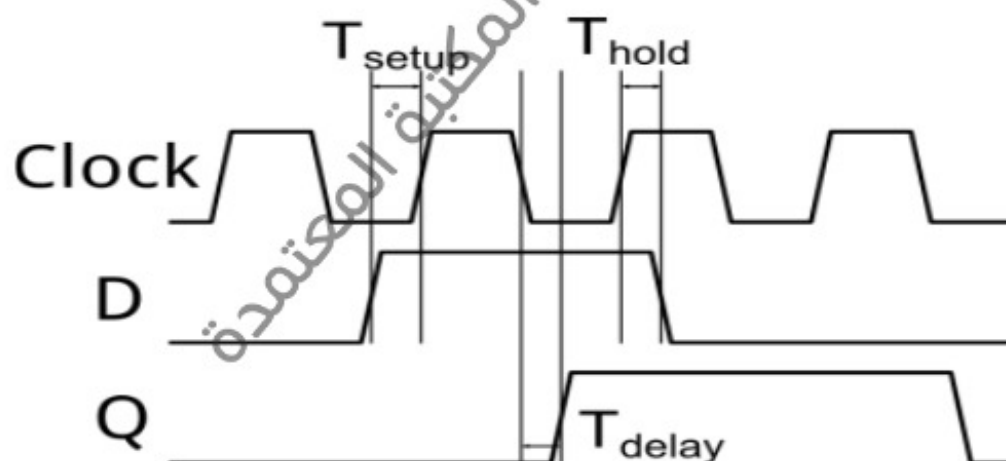


هو الزمن الذي يجب أن يبقى فيه دخل القلاب ثابتة قبل وصول النبضة، أي يجب أن تصل إشارة الدخل وتبقى ثابتة لفترة زمنية هي t_s قبل أن تأتي إشارة الـ CLK.

3. Hold time:



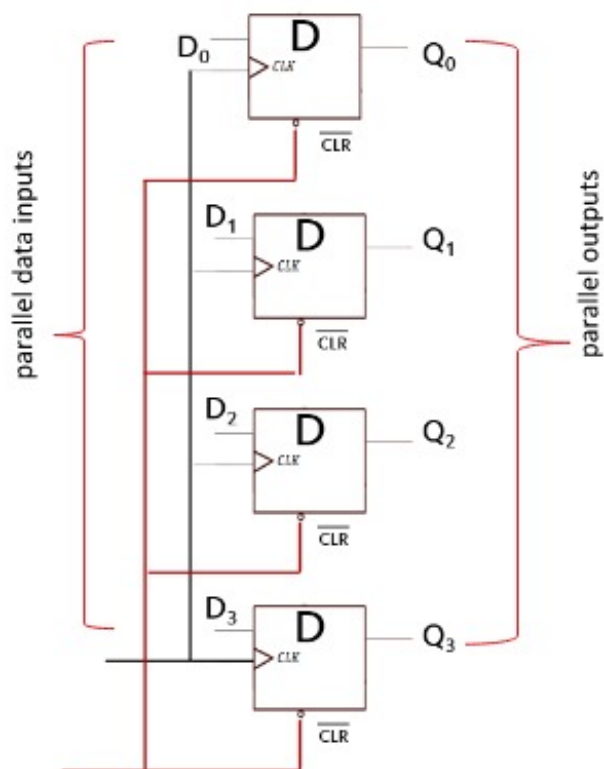
وهو أقل زمن يجب أن تثبت فيه قيم الدخل بعد وصول النبضة حتى نحصل على خرج صحيح.



صورة للتوضيح أيضاً:

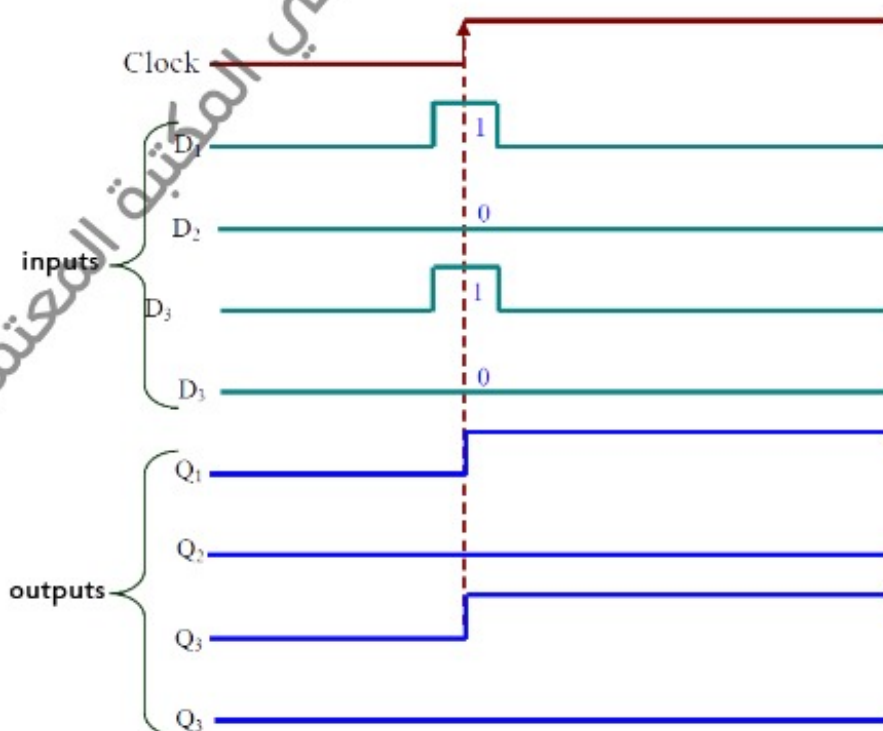
FF_application

1. parallel data storage:



لنفرض لدينا بيانات نريد تخزينها وهي 1010 لذا نطبق على المداخل D_1, D_2, D_3, D_4 هذه القيم المطلوب تخزينها وعند وصول أول نبضة فإننا نحصل على هذه القيم نفسها بالخرج، أي أنه كل ما هو مطبق على الدخل نحصل عليه في الخرج عند وصول أول نبضة.

وبالرجوع إلى المخطط الموضح أدناه نجد أن البيانات المراد تخزينها (1010) والتي تكون موجودة على المداخل عند وصول الجبهة الصاعدة لأول نبضة.

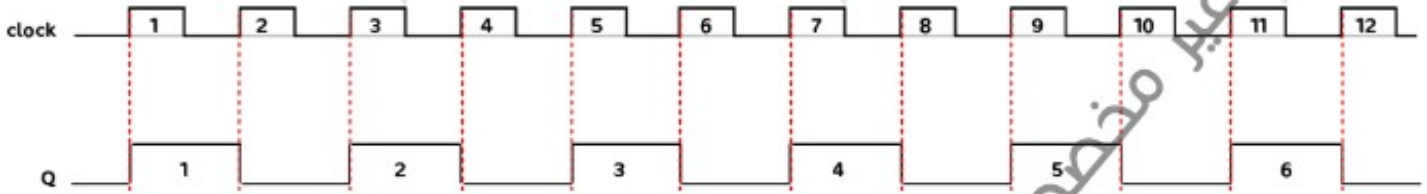
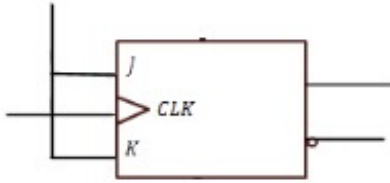


2. Frequency Division:

ليكن لدينا القلاب الآتي:

حيث $J = K = 1$.

نلاحظ أنه يعمل عند الجبهة الصاعدة سنعتبر بدايةً أن $Q = 0$.



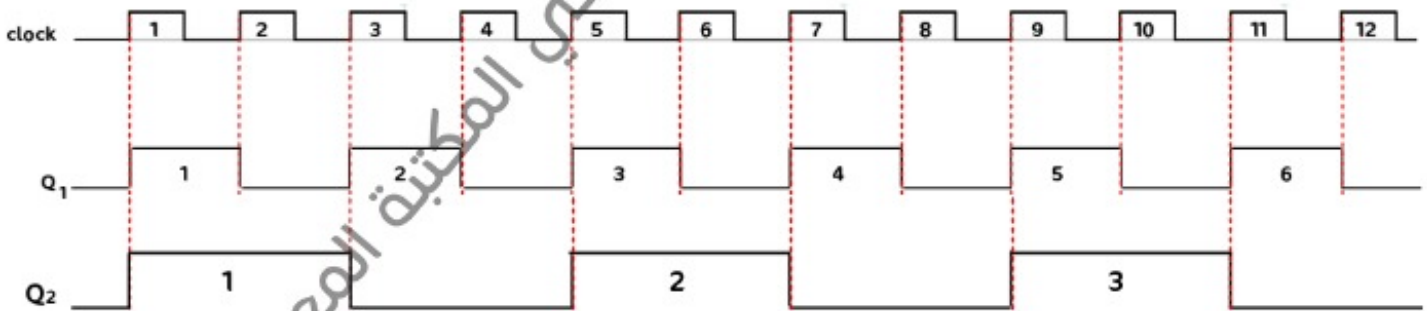
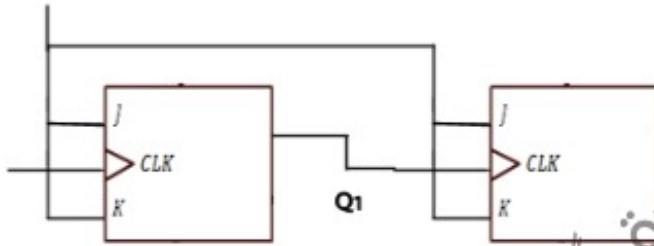
عند وصول الجبهة الصاعدة الاولى نلاحظ أن $J = K = 1$ وهذا ما يسمى بحالة Toggle عندها يكون Q^{t+1} يساوي عكس Q ومن ثم عند وصول الجبهة الصاعدة الثانية كذلك نفس الامر وهكذا إلى آخر قطار النبضات، نلاحظ بالنهاية أننا حصلنا على 6 نبضات بينما كان الدخل هو 12 نبضة لذلك نقول أنه قسّم قطار النبضات، أي أنه قسّم التردد.

ويمكن كذلك وصل قلابين كما يلي :

حيث يكون خرج الأول هو نبضة للثاني

وكذلك $J = K = 1$ لكل منهما، عندها يقوم الثاني

بتقسيم تردد الأول كما موضح بالمخطط:



Finally, it's the end of Logic circuit trip..

يكفي أنك تعبت وسعيت وحاولت، نراكم لاحقاً في الفصل القادم، بالتوفيق جميعاً.. 😊

دعواتكم..!